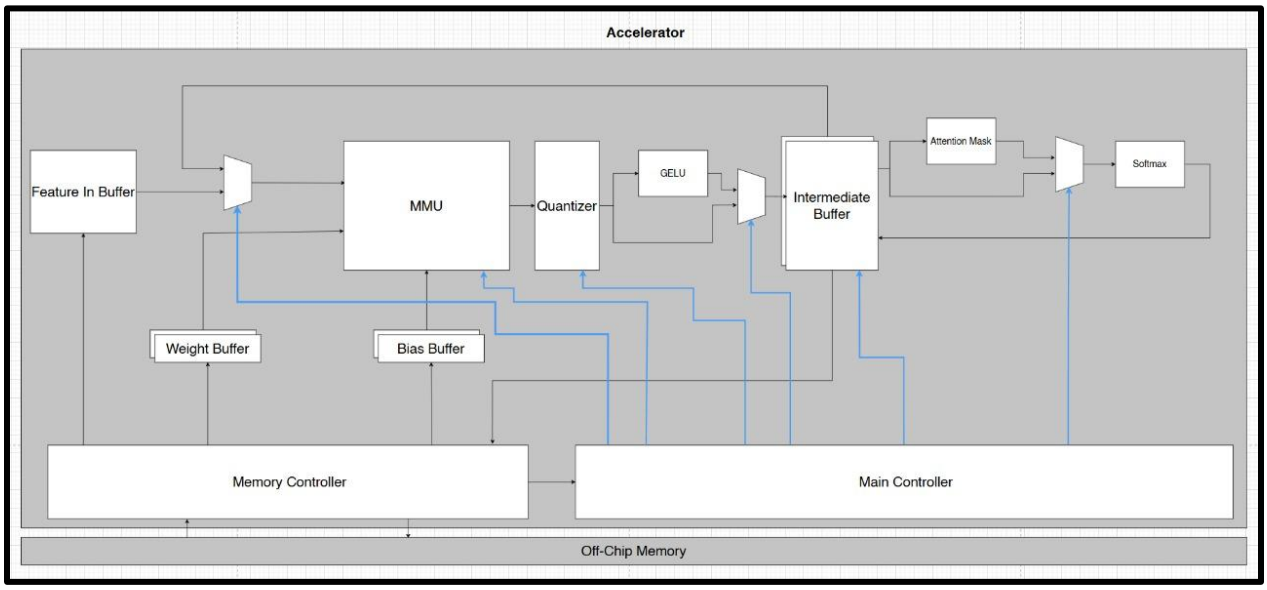
Architecture Diagrame



Main Controller States Description

Hardware Domain

1. IDLE State:

In this state the accelerator waiting for the Start signal from the host meaning that the Image are already loaded in the Feature in buffer and the next transition will be to `CONFIG_LOAD` stage.

2. CONFIG_LOAD State:

In this state the host needs to give the controller some givens required for the Swin-Transformer block like:

- How many stages (Basic Layers) the network has?
- The depth of each stage (number of Swin Blocks)?
- The feature dimension (embed dim) for each stage?
- The window size for attention?

Then after the done signal asserted by the host it will go to the next state which is `PE_LOAD_W` state.

3. PE_LOAD_W

In this state the control unit should order the memory controller to load the weights and the biases of the patch embedding stage into the weight and bias buffers, in order to use them in the PE process with the image stored in the feature in buffer, then it goes to the `PE_EXEC` when the memory controller send the done Flag.

Hardware controls & communication

In this state the main controller must communicate with the memory controller, to load the weights corresponding for the Patch embedding process in the buffers. so the expected control signals are:

The Buffers can hold the whole matrix	The Buffers can hold Tile from the matrix
1. **Start Signal.** 2. **Opcode,** indicating which data will be loaded from the off-chip memory. 3. **Done flag,** the main controller will wait for the done flag.	1. **Start Signal.** 2. **Opcode**, indicating which data will be loaded from the off-chip memory. 3. **Tile index** 4. **Done flag**, the main controller will wait for the done flag.

4. PE_EXEC

In this state the actual process of the PE begins, when I receive the done flag (of the intermediate buffers) it will go to the `BLOCK_START` state or the `PE_LOAD_W` based on which scenarios we have:

The Buffers can hold the whole matrix	The Buffers can hold Tile from the matrix
1. **Necessary Signals,** for the weight & Bias buffers, MMU, etc… (until we store the data in the intermediate buffer) 2. **Done flag,** the main controller will wait for the done flag, then it will go to the `BlOCK_START` state	1. **Necessary Signals,** for the weight & Bias buffers, MMU, etc… (until we store the data in the intermediate buffer) 2. **Done flag**, the main controller will wait for the done flag, then it will go to the `PE_LOAD_W` state

Hardware controls & communication

The communication must satisfy the following:

- Inputs image stored in the future buffers. (in terms of column/row).
- Weight buffers & bias buffers (in terms of row/column).

- Main controller must assert the start flag for the MMU unit to start the operation "**Convolution**".
- After the MMU block assert the done flag (for all the tiles) then the main controller will assert the quantization block start signal.
- After the quantization block assert the done signal, the main controller will choose the path that bypath the relu unit to store the output to the intermediate buffers.

5. BLOCK_START

In this stage the controller must check in the following scenarios to decide which state is the next one.

Scenarios:

- If the **current_layer_idx** and **current_block_idx** both are maximum (equal to the depth of the transformer and the swin stages per layer) this means that the final layer is finished so the next state should be **`WRITE_OUTPUT` state to write the output in the off ship memory and assert the start signal means that the software domain will start.**
- If the **current_layer_idx** are **not** maximum, While the **current_block_idx** are maximum. This means that the current layer is done but this is not the last layer so the next state will be `PM_LOAD_W` state.
- Any other case, it act as a transition until I reach those cases so the flow will complete as it is and the next state is **the based on the number of swin block** as it will be `MSA_LOOP_START` if the W-MSA will be executed or it will be `CYCLIC_SHIFT` if the SW-MSA will be executed.

6. CYCLIC_SHIFT

The controller enables the hardware logic responsible for cyclically shifting the feature map data within the `Intermediate Buffer`. It waits in this state until the shift operation is complete, then transitions to `MSA_LOOP_START`.

7. MSA_LOOP_START

This state initializes the MSA processing loop. It resets an internal counter, ` msa_step_counter `, to 0. It then transitions immediately to `MSA_DECIDE_ACTION`. As based on the value of ` msa_step_counter ` it will decide the operation in the MSA will be.

8. MSA_DECIDE_ACTION

This is the main dispatch state for the MSA loop. It checks the `msa_step_counter` to decide the next action, The operations based on this signal.

So based on the buffers sizes (it f it can store the Concatenated matrix) all at once I will only need to load them one time so the transition will be to `MSA_LOAD_W` state, before starting the operation.

Hardware controls & communication

The communication will depend on the PHASE number:

A. Phase (0,1,2), Generate (Q,K,V):

- Inputs data from the intermediate buffers. (in terms of column/row).
- Weight buffers & bias buffers (in terms of row/column).
- Main controller must assert the start flag for the MMU unit to start the operation "F**ully Connected**".
- After the MMU block assert the done flag (for all the tiles) then the main controller will assert the quantization block start signal.
- After the quantization block assert the done signal, the main controller will choose the path that bypath the relu unit to store the output to the intermediate buffers.

B. Phase 3, Calculate $Q \times K^t$

- Inputs data from the intermediate buffers. (in terms of column/row).
- Weight buffers & bias buffers (in terms of row/column).
- Main controller must assert the start flag for the MMU unit to start the operation "**QK_Transpose**" to calculate Q ×k^t
- After the MMU block asserts the done flag (for all the tiles) then the main controller will assert the quantization block start signal, for both paths.
- After the quantization block assert the done signal, the main controller will choose the path that bypath the relu unit to store the output to the intermediate buffers.

C. Phase 4, Calculate Softmax

- Inputs data from the intermediate buffers. (in terms of column/row).
- Main controller must the mux selection to bypath the attention mask block (for WMSA) while (for SWMSA) mux selection will choose the attention block path.

D. Phase 5, Calculate $S \times V$

- Inputs data from the intermediate buffers. (S & V).
- Main controller must assert the start flag for the MMU unit to start the operation "**Score × V**" to calculate Q ×k^t
- After the MMU block assert the done flag (for all the tiles) then the main controller will assert the quantization block start signal, for both paths.
- After the quantization block assert the done signal, the main controller will choose the path that bypath the relu unit to store the output to the intermediate buffers.

9. STATE_MSA_LOAD_W

This state issues the specific command to the `Memory Controller` to load the correct set of weights/biases then goes to`STATE_MSA_EXEC`.

Hardware controls & communication

In this state the main controller must communicate with the memory controller, to load the weights corresponding for the MSA process in the buffers.

10. STATE_MSA_EXEC

Based on `msa_step_phase`, this state sets the appropriate `msa_opcode` and provides it to the `MSA_FSM`. It then asserts the `MSA_FSM_start` signal to trigger the subordinate compute engine. It transitions to `MSA_DEVICE_ACTION` when the `MSA_FSM` assert the done flag (return to idle).

11. STATE_REVERSE_SHIFT

(SW-MSA Only) After the MSA loop is complete, this state enables the hardware to reverse the cyclic shift. It waits for completion, then transitions to `STATE_MLP_LOAD_W`. For W-MSA blocks, this step is bypassed.

12. STATE_MLP_LOAD_W

Issues a command to the `Memory Controller` to load the weights and bias for the first linear layer of the MLP block. this will be followed by another load for the second linear layer.

13. STATE_MLP_EXEC

The controller triggers the subordinate `MLP_FSM` to start processing. And when the sub FSM assert the done flag it returns to `Block_START`.

14. STATE_PM_LOAD_W

Issues a command to the `Memory Controller` to load the weights for the `Patch Merging` linear layer into the `Weight Buffer`. It then transitions to ` STATE_PM_EXEC`.

15. STATE_PM_EXEC

The controller configures the Datapath for the patch merging operation. This involves controlling the addressing to read features from a 2x2 neighborhood, concatenating them, and then feeding them to the MMU.

Hardware controls & communication

The communication must satisfy the following:

- Input stored in the intermediate buffers. (in terms of column/row).
- Weight buffers & bias buffers (in terms of row/column).
- Main controller must assert the start flag for the MMU unit to start the operation "**Convolution**".
- After the MMU block asserts the done flag (for all the tiles) then the main controller will assert the quantization block start signal.

After the quantization block assert the done signal, the main controller will choose the path that bypath the relu unit to store the output to the intermediate buffers

16. STATE_WRITE_OUTPUT

Issues a command to the `Memory Controller` to write the final result vector from the output buffer to `Off-Chip Memory`.

17. FINISH

This state signals to the `IDLE state` that the assigned task is complete.

Description of MSA States

1. MSA_IDLE

The default state. The FSM is idle and waits for the `MSA_FSM_start` signal from the `Main Controller`. When the start signal is received, it captures the `msa_opcode` provided by the `Main Controller` and, based on the opcode, transitions to either `MSA_EXEC_MMU` or `MSA_EXEC_SOFTMAX`.

2. MSA_EXEC_MMU

This state is entered for any MMU-based operation (e.g., `OP_GEN_Q`, `OP_QKT`, `OP_ATTN_V`). It performs the specific matrix multiplication task commanded by the current `msa_opcode`, The communication and the hardware are discussed in the `MSA_DECIDE_ACTION` state.

3. MSA_EXEC_SOFTMAX

This state is entered only when the `msa_opcode` is `OP_SOFTMAX`. It orchestrates the flow through the attention mask and softmax hardware blocks.

4. MSA_FINISH

This state signals to the `Main Controller` that the assigned task is complete.

Hardware & Control Signals

- Asserts the `MSA_FSM_done` signal for one clock cycle.
- On the same cycle, it unconditionally transitions back to `MSA_IDLE` to await the next command.

Description of MLP States

1. MLP_IDLE

The default state. The FSM waits for the `MLP_FSM_start` signal. When received, it assumes the weights for the first linear layer are loaded and transitions to `MLP_L1_EXEC`.

2. MLP_L1_EXEC

This state executes the first linear layer of the MLP block.

Hardware controls & communication

The communication must satisfy the following:

- Input stored in the intermediate buffers. (in terms of column/row).
- Weight buffers & bias buffers (in terms of row/column).
- Main controller must assert the start flag for the MMU unit to start the operation "**Fully Connected**".
- After the MMU block asserts the done flag (for all the tiles) then the main controller will assert the quantization block start signal.
- After the quantization block assert the done signal, the main controller will choose the path that Contain the RELU unit to store the output to the intermediate buffers.

3. MLP_SIGNAL_FOR_W2

This key state informs the `Main Controller` that it is ready for the second linear layer's weights.

4. MLP_L2_EXEC

This state executes the second linear layer of the MLP block. Hence for the the hardware controls and the communication are same as state `MLP_L1_EXEC`

5. MLP_RESIDUAL_ADD

This state performs the final residual connection for the entire Swin Block.

6. MLP_FINISH

This state signals to the `Main Controller` that the entire MLP block, including both linear layers and the residual connection, is complete.

Software Domain

1. STATE_POOL_EXEC

Configures and starts the global average pooling hardware on the final feature map. Transitions to ` STATE_FC_LOAD_W` .

2. STATE_FC_LOAD_W

Issues a command to the `Memory Controller` to load the weights for the final fully-connected (classifier) layer.

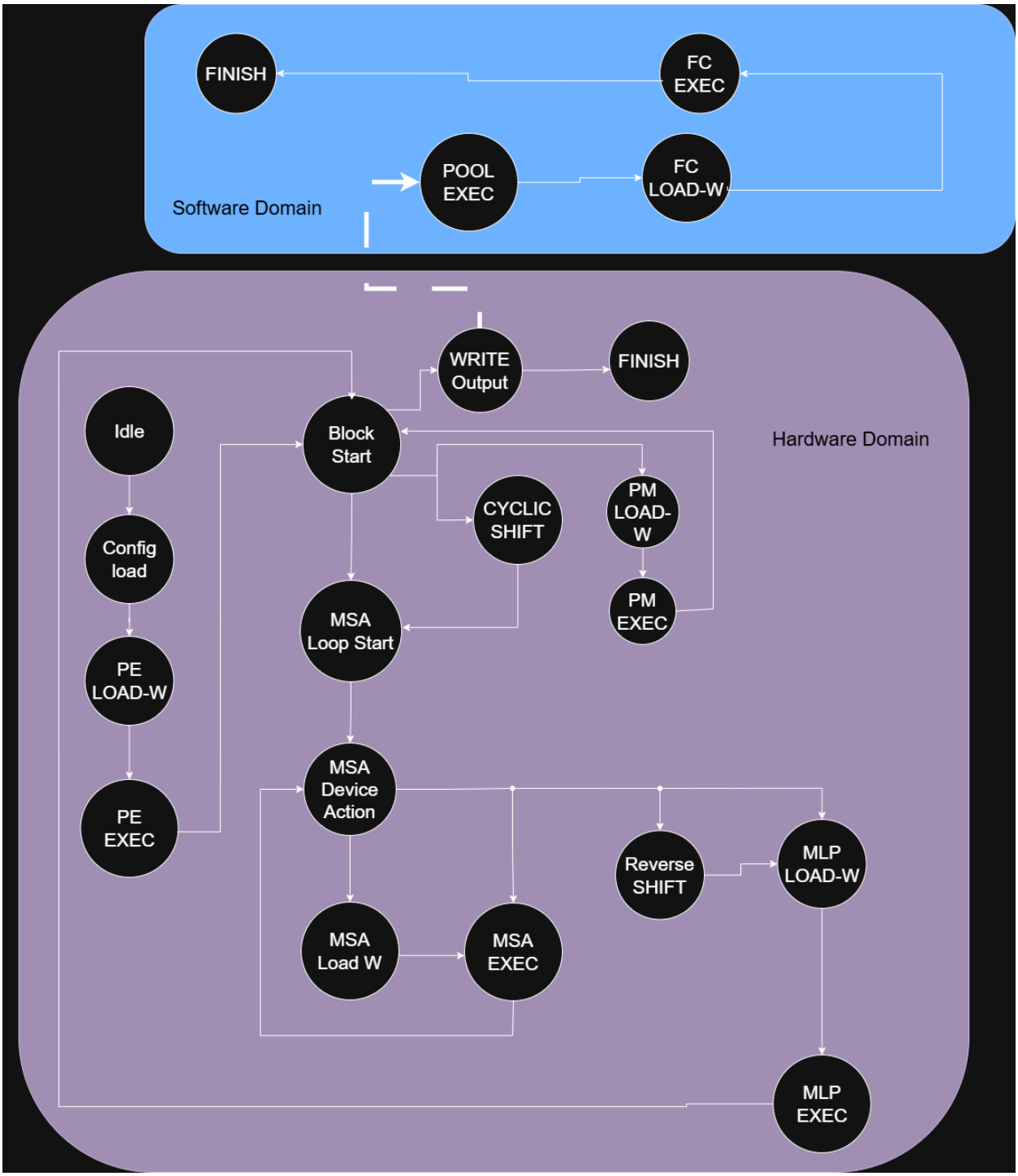
3. STATE_FC_EXEC

Configures and starts the `MMU` for the final classification computation. Transitions to `STATE_FC_WAIT_EXEC` .

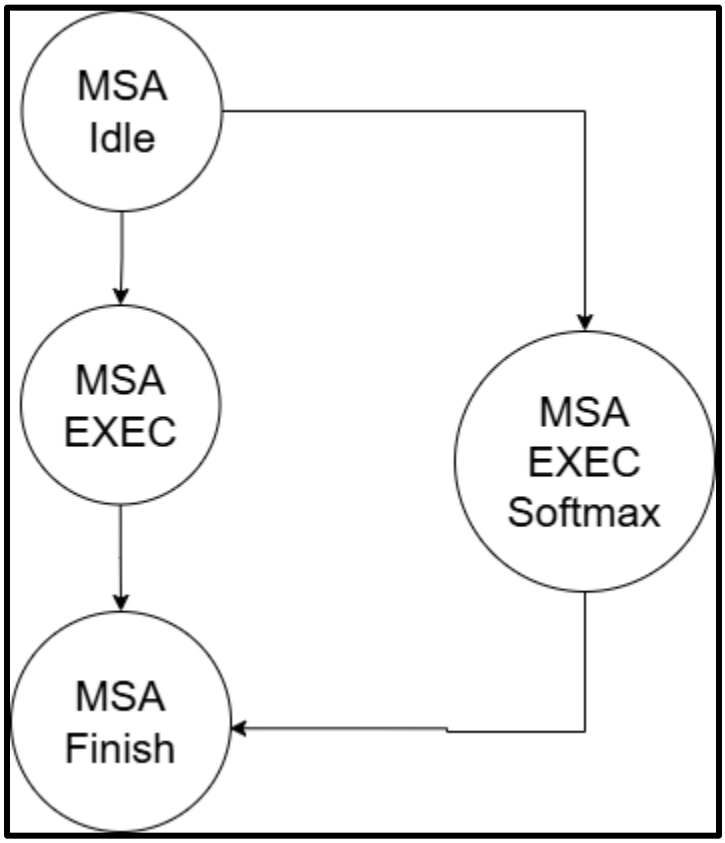
4. STATE_FINISH

Asserts the `processing_done` signal to notify the host system that the inference is complete. Transitions back to `STATE_IDLE` to await the next task.

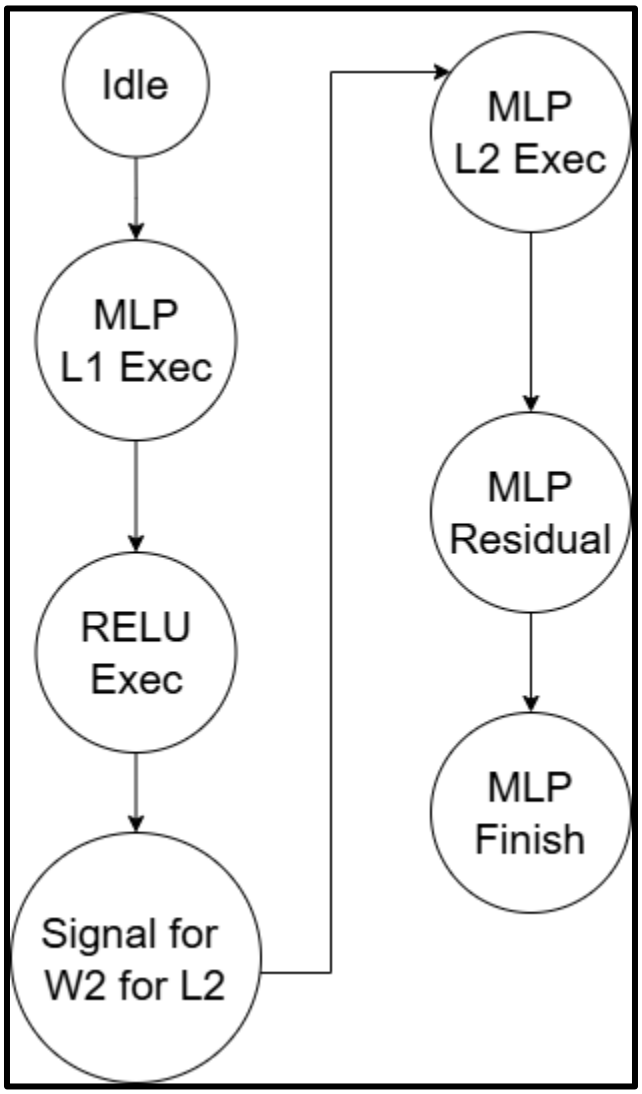
Main Controller State Diagrame



MSA State Diagrame



MLP State Diagrame



Control Signals of the Arch. Block

1. ### MMU:

   Outputs:

   - Start Signal: also indicate the validation of the data.
   - Opcode signal: indicate the operation will be done. (**4 bits**)

   Inputs:

   - Done Signal: indicate the validation of the output, also finishing of the operation

2. ### Memory Controller:

   Outputs:

   - Start Signal: Tells the memory to start the loading of the kernel.
   - Opcode signal: which weights will be loaded.
   - Tile index: which tile will be loaded.

   Inputs:

   - Done Signal: indicate the validation of the output, also finishing of the operation.

3. ### Softmax:

   Outputs:

   - Start Signal: also indicate the validation of the data.

   Inputs:

   - Done Signal: indicate the validation of the output, also finishing of the operation

4. ### Buffers:

   Outputs:

   - Start Signal: also indicate the validation of the data.

5. ### Quantizer

   Outputs:

   Inputs: